

LIVRABLE Q4 : MODÉLISATION DE LA SOLUTION SHOPWISE

Projet : Plateforme SaaS de digitalisation pour commerces de proximité

Objectif : Définir l'architecture logicielle et les flux de données pour garantir performance, sécurité et maintenabilité.

1. CHOIX DE L'ARCHITECTURE GLOBALE

Pour répondre au budget de 10 000 € tout en assurant une évolution vers du multi-commerces, nous optons pour une **Architecture Client-Serveur basée sur une API REST** avec un déploiement de type **Modular Monolith** (Monolithe Modulaire).

- **Pourquoi pas des microservices ?** Trop coûteux en infrastructure et en maintenance pour un budget de 10k€.
- **Pourquoi le Monolithe Modulaire ?** Il permet de séparer proprement les domaines (Stocks, Clients, RDV) dans le code, facilitant une future transition vers des microservices si ShopWise explose, tout en gardant des coûts d'hébergement faibles au départ.

2. MODÉLISATION FONCTIONNELLE (DIAGRAMME DE COMPOSANTS UML)

Note pour ton document : Tu devrais réaliser ce schéma sur un outil comme Lucidchart ou Draw.io. Voici la structure à représenter :

Les 3 Couches de l'application :

1. **Couche Présentation (Frontend) :**
 - **PWA (Progressive Web App)** : Développée en **React.js** ou **Vue.js**.
 - *Justification* : La PWA permet d'avoir une seule base de code pour le Web (PC de Marie) et le Mobile (smartphone Android), réduisant les coûts de développement de 40% par rapport à du natif.
2. **Couche Application (Backend API) :**
 - **Runtime** : Node.js avec Express (pour la rapidité de développement).
 - **Modules** : **StockManager**, **BookingEngine**, **LoyaltyService**, **AuthService**.
3. **Couche Données (Persistence) :**
 - **Base de données** : PostgreSQL (Relationnel).
 - *Justification* : Indispensable pour garantir l'intégrité des transactions (stocks et ventes) via les propriétés ACID.

3. FLUX DE DONNÉES ET INTERACTION

Prenons l'exemple critique du **Flux d'alerte de stock bas** :

1. Le module **Vente** enregistre une transaction.
2. Le composant **StockManager** décrémente la quantité en BDD.
3. Un **Observer** (Pattern de conception) surveille le seuil critique.
4. Si le seuil est atteint, le **NotificationService** est appelé pour envoyer un SMS/Email via une API tierce (type Twilio ou Brevo).

4. JUSTIFICATIONS TECHNIQUES APPROFONDIES (L'OEIL DE L'EXPERT)

A. Scalabilité et Multi-tenancy

Pour que ShopWise puisse accueillir des milliers de commerçants, nous utilisons une stratégie de **cloisonnement des données par Tenant_ID**. Chaque table en base de données possède une colonne **shop_id**.

- **Avantage** : Coût d'infrastructure minimal (une seule instance de BDD pour tous les clients au début).

B. Respect du RGPD "By Design"

- **Chiffrement** : Les données sensibles (noms, emails, téléphones des clients de Marie) sont chiffrées au repos (AES-256).
- **Module de Consentement** : Intégration d'un flag **gdpr_consent** avec horodatage pour chaque client dans le module de fidélité.
- **Isolation** : Un commerçant A ne peut techniquement jamais requêter les données d'un commerçant B grâce au middleware d'authentification qui injecte le **shop_id** dans chaque requête SQL.

C. Accessibilité et Contraintes Matérielles

L'interface sera conçue selon le principe du **Mobile-First** mais optimisée pour les navigateurs légers.

- Utilisation de **Lazy Loading** pour que l'application reste fluide même sur l'ancien PC Windows de Marie.
- Mode hors-ligne partiel (via Service Workers de la PWA) pour permettre la consultation du stock même en cas de coupure internet temporaire dans la boutique.

5. STACK TECHNIQUE PRÉCONISÉE

Composant	Technologie	Justification
Frontend	React + Tailwind CSS	Rapidité de développement et interface ultra-légère.

Backend	Node.js / TypeScript	Typage fort pour limiter les bugs et excellente performance I/O.
BDD	PostgreSQL (Hébergé)	Fiabilité des données et gestion facile des relations complexes.
Hébergement	Docker sur un VPS (ex: OVH ou DigitalOcean)	Équilibre parfait entre coût (10-20€/mois) et flexibilité.